hope
CMS

Customer Management System

MEMBERS

Frinz Hughwie D. Bautista

John Pete P. Casapao

Lorenzo Nheo M. Queñano

Kim G. Moguer

Ralph Anthony B. Biazon

What is HopeCMS?

React + Supabase SaaS web app
Built on Vite, TailwindCSS, React Router

Role-based access control
3 user types × 9 granular rights

Full customer lifecycle
CRUD with soft-delete & recovery

Sales drill-down
Customer → Transaction → Line Items

Reports & Analytics
Top customers, product revenue

Admin module
User activation, role management

Table Relationships & ERD

Core database schema with Supabase PostgreSQL

user
userid PK
username
email
user_type
record_status
is_superadmin

customer
custno PK
custname
address
payterm
record_status
stamp

sales
transno PK
custno FK
salesdate
empno

salesdetail
transno FK
prodcode FK
quantity

UserModule_Rights
userid FK
rightcode
right_value

product
prodcode PK
description
unit

pricehist
prodcode FK
effdate
unitprice

Primary Key (PK)

Foreign Key (FK)

Regular Column

Rights stored in UserModule_Rights table. SUPERADMIN rights sync automatically via DB trigger on role change.

Rights Matrix

3 user types × 9 granular permission codes

	USER	ADMIN	SUPERADMIN
CUST_VIEW View Customers	✓	✓	✓
CUST_ADD Add Customer	—	✓	✓
CUST_EDIT Edit Customer	—	✓	✓
CUST_DEL Delete Customer	—	✓	✓
SALES_VIEW View Sales	✓	✓	✓
SD_VIEW Soft-Delete View	—	✓	✓
PROD_VIEW View Products	✓	✓	✓
PRICE_VIEW View Prices	—	✓	✓
ADM_USER Admin Module	—	—	✓

Authentication Flow

Email/Password + Google OAuth · Active-account guard · Token refresh

Email / Password

1. User submits Login form
2. supabase.auth.signInWithPassword()
3. Fetch user row from 'user' table
4. Check record_status === 'ACTIVE'
5. INACTIVE → signOut() + error message
6. ACTIVE → setCurrentUser() → Dashboard

Google OAuth 2.0

1. User clicks 'Sign in with Google'
2. signInWithOAuth({ provider: 'google' })
3. Redirect to Google consent screen
4. Callback: /auth/callback route fires
5. AuthCallbackPage fetches session
6. Same active-guard check → Dashboard

Customer CRUD

Create · Read · Update · Delete (soft) — with role-gated controls

Read

CUST_VIEW

List all active customers. ADMIN/SUPERADMIN also sees soft-deleted rows.

Create

CUST_ADD

Modal form triggers addCustomer() with stamp = 'CREATED by <uid> on <date>'.

Update

CUST_EDIT

EditCustomerModal calls updateCustomer() — stamp updated to UPDATED.

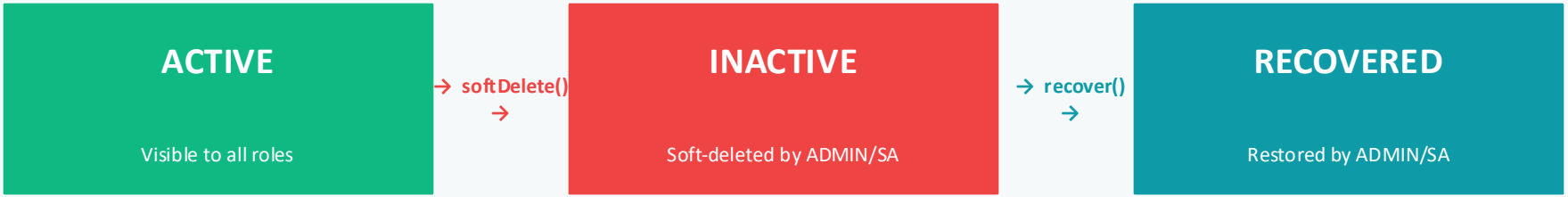
Delete

CUST_DEL

Soft-delete only — record_status → INACTIVE. Never a hard DELETE.

Soft-Delete & Recovery

Data is never permanently deleted — record_status drives visibility



softDeleteCustomer()

- Sets record_status = 'INACTIVE'
- Writes stamp: DEACTIVATED by … on …
- Customer disappears from USER view
- Sales history is fully preserved
- Requires CUST_DEL right

recoverCustomer()

- Sets record_status = 'ACTIVE'
- Writes stamp: REACTIVATED by … on …
- Available on DeletedCustomersPage
- Requires CUST_DEL + SD_VIEW rights
- One-click recovery from deleted list

Why Soft-Delete?

- Preserves referential integrity
- Audit trail via stamp field
- Reversible — no accidental data loss
- ADMIN/SA can audit all records
- Supabase RLS enforces visibility

Sales Drill-Down Demo

Customer → Transaction → Line Items — powered by Supabase nested selects

LEVEL 1

Customer

CustomersPage lists all active customers. Click a customer row to drill in.

FIELDS RETURNED

- custno
- custname
- address
- payterm

LEVEL 2

Transactions

CustomerDetailPage → getSalesByCustomer(custno) fetches all transactions.

FIELDS RETURNED

- transno
- salesdate
- empno

LEVEL 3

Line Items

Click a transaction → getSalesDetail(transno) with product join.

FIELDS RETURNED

- prodcode
- description
- unit
- quantity

Reports Module

3 report views backed by Supabase database views

Customer Sales Summary

VIEW

`customer_sales_summary`

PAGE COMPONENT

`CustomerSalesSummaryPage`

COLUMNS

- custno
- custname
- payterm
- record_status
- totalTransactions
- totalSpend

Ordered by totalSpend DESC

Top 10 Customers

VIEW

`customer_sales_summary`

PAGE COMPONENT

`TopCustomersPage`

COLUMNS

- custno
- custname
- totalTransactions
- totalSpend
- lastSaleDate

LIMIT 10 — highest spenders

Product Revenue

VIEW

`product_revenue`

PAGE COMPONENT

`ProductRevenuePage`

COLUMNS

- prodcode
- description
- unit
- total_revenue
- units_sold

Aggregated from salesdetail × pricehist

Admin Module

User activation · Role assignment · Superadmin self-recovery

User Activation

New registrations default to INACTIVE. ADMIN manually activates them via Activate button. Inactive users are denied login at the guard layer.

Role Management

SUPERADMIN can promote/demote users between USER and ADMIN roles. Each change triggers sync_admin_rights() DB trigger, updating UserModule_Rights automatically.

SUPERADMIN Guard

SUPERADMIN rows are protected — hidden from all mutation operations. Only the permanent is_superadmin flag holder can self-restore via AppShell recovery modal.

Architecture Overview

React SPA · Supabase BaaS · Vercel Edge Hosting

Frontend (Client)

- React 18 + Vite
- React Router v6
- TailwindCSS
- Context API (Auth + Rights)
- react-icons

Auth Layer

- Supabase Auth
- Email/Password
- Google OAuth 2.0
- JWT Sessions
- onAuthStateChange

Database (PostgreSQL)

- 7 core tables
- 2 database views
- Row-Level Security
- sync_admin_rights trigger
- Supabase PostgREST API

DEPLOYMENT

- Vercel (CI/CD auto-deploy from GitHub main branch)
- Environment variables: VITE_SUPABASE_URL + VITE_SUPABASE_ANON_KEY
- Supabase project: hopecms (ap-southeast-1 region)
- Vitest unit tests: 4 sprint-1 auth flow specs

RLS & Security Highlights

Defense-in-depth: App layer · RLS policies · DB triggers

Row-Level Security

- ✓ Every table has RLS enabled
- ✓ SUPERADMIN rows blocked from mutation
- ✓ Policies filter rows by auth.uid()
- ✓ App-level guard + RLS = double lock
- ✓ INACTIVE accounts see 0 rows

INACTIVE Guard

- ✓ signInWithPassword checks db user
- ✓ Error surfaced cleanly to UI
- ✓ record_status !== 'ACTIVE' → signOut()
- ✓ No raw Supabase errors exposed
- ✓ Blocks pending/deactivated accounts

Role Sync Trigger

- ✓ sync_admin_rights() fires on UPDATE
- ✓ No manual rights management needed
- ✓ Detects user_type change
- ✓ SUPERADMIN always gets all 9 = 1
- ✓ Writes correct 9-right profile to DB

JWT + OAuth

- ✓ Supabase issues signed JWT tokens
- ✓ signOut clears all sb-* localStorage keys
- ✓ Google OAuth with prompt: select_account
- ✓ Redirect URL whitelisted in Supabase
- ✓ Tokens refreshed via onAuthStateChange

Lessons Learned

RLS is non-negotiable

Relying on app-layer checks alone is fragile. Database-level RLS policies provide the security floor.

Soft-delete by design

Designing for data recovery from the start prevented major refactors. record_status is now core to every query.

Context API vs. Redux

For this scale, React Context (AuthContext + UserRightsContext) was cleaner and faster than a full state library.

Stamp auditing

The stamp VARCHAR(60) pattern gives us a lightweight audit trail without a full audit_log table.

OAuth edge cases

Google OAuth required careful handling: redirects, callback routes, and session sync needed 3 separate fixes.

THANK YOU

Built with

- React
- Supabase
- Vite
- Vercel